

their memory — ensuring developers don't have to manually allocate or deallocate memory like in C or C++.

The result:

- No memory leaks (in most cases)
- No dangling pointers
- Safe concurrency without manual memory management headaches

Let's break Go's garbage collection strategy into clear components.

Tricolor marking algorithm: Go's GC uses a tricolor

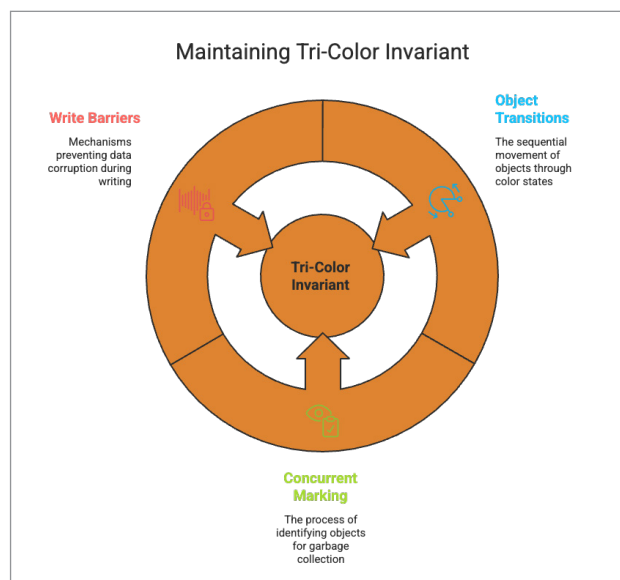


Figure 4: Tricolor

Go GC: Strengths vs weaknesses

Aspects	Strengths	Weaknesses
Concurrent collection	Runs GC alongside the application, reducing pauses and maintaining responsiveness	May lead to less control over when and how GC occurs compared to Java
Low-latency goal	Sub-millisecond pause times, ideal for performance-critical applications	Longer or more frequent pauses may still happen in high-load scenarios
Automatic tuning	Minimal configuration required from developers	Less fine-grained control for tuning memory management
Evolving technology	Continuous improvements since Go 1.5, making it more efficient with each release	Still not as mature as Java's GC options and may not fit all use cases

marking abstraction to track object reachability. Objects are conceptually coloured:

- **White:** Unreachable (candidate for garbage)
- **Gray:** Reachable but not yet fully scanned
- **Black:** Reachable and fully scanned

The collector traverses all reachable objects starting from roots (global variables, stack pointers, etc), moving objects from gray to black, and eventually clearing away white (garbage) objects.

Concurrent mark and sweep:

- **Mark phase:** Runs concurrently with application goroutines.
- **Sweep phase:** Also incremental and concurrent, freeing memory progressively.
- Minimal stop-the-world pause is required at the start and end to ensure safe root marking and clean-up.

Write barrier (tricolor invariant maintenance): To safely run GC concurrently with mutator threads (the running program), Go uses a write barrier — a mechanism that intercepts pointer writes to ensure the tricolor invariant isn't violated. This guarantees correctness even as the program continues to modify memory during a GC cycle.

Go's GC is ideal for:

- Web services
- Cloud infrastructure
- Containerised environments

Julia's garbage collection strategy

Julia uses a mark-and-sweep garbage collector, which

Julia GC: Strengths vs weaknesses

Aspects	Strengths	Weaknesses
GC design	Optimised for short-lived objects, typical in scientific computing	Stop-the-world GC with high pause times can be disruptive in real-time apps
Tied to JIT compilation	Strong integration with Julia's JIT execution model for numerical computing	GC performance and behaviour are tightly linked to JIT, limiting flexibility
Tuning and control	Simple memory management with less overhead for developers	Fewer tunable options compared to Go and Java, limiting optimisation choices
Use case optimisation	Excellent for numerical computing, data science, and machine learning	Not ideal for latency-sensitive applications or long-running services